

Cross-validation and nested cross-validation

Stefano Diciotti¹

¹Dipartimento di Ingegneria dell'Energia Elettrica e dell'Informazione "Guglielmo Marconi" - DEI, Università di Bologna

What is K-Fold Cross-Validation?

- K-Fold Cross-Validation is a technique to estimate a model's generalization error.
- The dataset is split into k equal-sized folds.
- For each fold:
 - One fold is used as the test set.
 - The remaining $k - 1$ folds are used as the training set.
- This process is repeated k times, with each fold used exactly once as test.

Important note:

- Hold-out strategies can become inefficient when the available dataset is relatively small, because splitting the data further reduces the amount available for training and test.

10-Fold Cross-Validation Example

Specific case: $k = 10$

- The dataset is split randomly into 10 folds of approximately equal size.
- We perform 10 training-test cycles:
 - Each time, 9 folds are used for training, and 1 fold is used for test.
- Final model performance is estimated by averaging the validation results over all 10 folds.



Why Use K-Fold Cross-Validation?

Advantages:

- More reliable estimate of generalization error compared to a single validation set.
- Makes efficient use of limited data: every example is used both for training and validation.
- Reduces variance in the performance estimate compared to a single hold-out split.

Typical choice of k :

- Statistical studies suggest setting $k = 5$ or $k = 10$ for a good balance between bias and variance.

Special case: Leave-One-Out Cross-Validation (LOOCV)

- $k = m$ (number of examples).
- Each fold contains exactly one example for validation.
- Provides an almost unbiased estimate of generalization error, but with high variance and high computational cost.

Motivation:

- Standard K-Fold Cross-Validation can lead to optimistic bias when hyperparameters are tuned using the same data used for evaluation.
- Nested Cross-Validation provides an unbiased estimate of generalization error when hyperparameter tuning is involved.

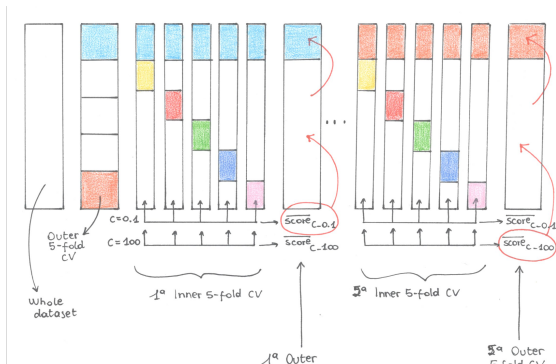
Key idea:

- Separate the process of hyperparameter selection and model evaluation by using two cross-validation loops:
 - An inner loop for hyperparameter tuning.
 - An outer loop for evaluating generalization error.

Nested Cross-Validation: Procedure

Steps:

- Outer loop:
 - Split data into k outer folds.
 - For each outer fold:
 - Hold it out as the test set for evaluation.
 - Use the remaining $k - 1$ folds to perform hyperparameter tuning (inner loop).
- Inner loop:
 - Perform cross-validation inside the training set to select the best hyperparameters.
- Final evaluation:
 - Train the model with the selected hyperparameters on the inner training folds and evaluate it on the outer fold.
 - Average the performance across outer folds to estimate generalization error.



Final Model After Nested Cross-Validation

After completing Nested Cross-Validation:

- We obtain an unbiased estimate of the expected generalization error.
- This estimate is based on the outer test folds.

To obtain the final model:

- Perform a new cross-validation over the entire dataset to select the best hyperparameters (equivalent to the inner loop).
- Retrain the model on the entire dataset using the selected optimal hyperparameters.
- No separate test set remains: the model's expected performance is the one estimated by the Nested CV.

Key point:

- The final model is trained on all available data, maximizing its potential while keeping the estimated generalization error unbiased.

Why Use Nested Cross-Validation?

Advantages:

- Provides an unbiased estimate of generalization performance, even when hyperparameters are tuned.
- Reduces the risk of optimistic bias caused by overfitting to the validation set.
- Ensures that the test set remains completely untouched during model selection and training.

Trade-offs:

- Higher computational cost: multiple models must be trained for each outer fold.

Choosing k in Nested Cross-Validation

Recommended choices:

- Statistical studies suggest setting $k = 5$ or $k = 10$.
- These values provide a good trade-off between:
 - Low bias (accurate estimate of generalization error).
 - Acceptable variance and computational cost.

Special case: Nested Leave-One-Out (Nested LOO)

- If $k = m$ (where m is the number of instances), we perform a Nested Leave-One-Out cross-validation.
- Every fold contains exactly one example for validation/test in both the inner and outer loops.
- This yields almost unbiased estimates but with very high computational cost and variance.

Cross-Validation:

Train → Evaluate on Test Set

Nested Cross-Validation:

Outer Loop: Train on Training Set → Inner Loop: Tune Hyperparameters

→ Evaluate on Outer Test Set

Key difference:

- In CV, no separation between hyperparameter tuning and evaluation: using the same data for both steps may cause optimistic bias.
- In nested CV, hyperparameter tuning and evaluation are strictly separated, leading to an unbiased estimate of generalization error.

Nested CV: Model Comparison and Feature Selection

Applications of Nested CV:

- Model comparison:
 - Use the inner cross-validation to select the best model among several candidates (e.g., SVM, Random Forest, Neural Network).
 - Use the outer cross-validation to evaluate the generalization performance of the selected model.
- Feature selection:
 - Use the inner cross-validation to select the best subset of features.
 - Use the outer cross-validation to estimate the generalization error after feature selection.

Key idea:

- The inner loop is used for making choices (model selection, feature selection, hyperparameter tuning).
- The outer loop is used only for an unbiased evaluation of the final decision.

Stratified Cross-Validation and Stratified Nested CV

Stratified Cross-Validation (CV):

- In standard CV, data splits are random and may cause imbalanced class distributions.
- Stratified CV ensures that each fold maintains approximately the same class proportions as the full dataset.
- Particularly important for imbalanced datasets.

Stratified Nested CV:

- Both the outer and inner splits are stratified.
- Preserves class balance during:
 - Hyperparameter tuning (inner CV)
 - Final model evaluation (outer CV)
- Reduces variance and provides a more reliable estimate of generalization performance.

- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, Chapter 5

<https://www.deeplearningbook.org/>